DATA ENCODING

Overview

- What is data encoding and why do we do it?
- CSV
- JSON
- YAML

Learning Objectives

- To be able to explain what data encoding is and why we do it
- To gain an understanding of what CSV, YAML & JSON are

What is Data Encoding?

- Encoding is the process of converting data into a specified format.
- Decoding is the reverse process - to extract information from the converted format.
- Common formats:

- JPG, MP4, AVI
- Morse code, Braille
- JSON, XML, YAML, CSV
- Analog, Digital

Why Do We Encode Data?

- Easier to store for computers (humans work with text, computers work with bytes)
- Removes redundancies from data (such as whitespace) so data size decreases
- Smaller data means it's more efficient to store and retrieve data

CSV (Comma Separated Values)

- A plain text file that uses specific structuring to arrange tabular data
- Only contains text data
- Each line of the file is a data record
- Is separated by a *delimiter* (comma, colon, tab etc.)

```
first_name, last_name, age
John,       Smith,     20
Sally,      Bloggs,    30
```

CSV - Dealing With Commas

What about data that contains a comma?
Well, in that scenario, we quote the data

```
first_name, last_name, age, test_scores
John,       Smith,     20,  "80, 76, 92"
Sally,      Bloggs,    30,  "72, 84, 90"
```

CSV in Python

Luckily for us, Python has its own csv library to read and write to/from CSV files.

https://docs.python.org/3/library/csv.html

CSV - Reading a File

Looking back to the previous module, opening a CSV can be done in a few lines like so:

```python
import csv

with open('pirates.csv', 'r') as file:
    reader = csv.reader(file, delimiter=',')
    for row in reader:
        print(row)
```

```
['name', 'ship', 'bounty', 'rank']
['Jack Sparrow', 'Black Pearl', '500000', 'Captain']
['Hector Barbossa', "Queen Anne's Revenge", '750000', 'Captain']
['Davy Jones', 'The Flying Dutchman', '1000000', 'Captain']
['Will Turner', 'The Flying Dutchman', '300000', 'First Mate']
['Elizabeth Swann', 'Interceptor', '200000', 'Captain']
['Joshamee Gibbs', 'Black Pearl', '100000', 'First Mate']
['Cutler Beckett', 'HMS Endeavour', '0', 'Commodore']
```

`reader` is a function in the CSV library which returns an object which will iterate over the lines in the given file

Reading to a Dictionary

We can read our CSV directly into a dictionary using `DictReader`:

```python
import csv

with open('pirates.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

```
{'name': 'Jack Sparrow', 'ship': 'Black Pearl', 'bounty': '500000', 'rank': 'Captain
{'name': 'Hector Barbossa', 'ship': "Queen Anne's Revenge", 'bounty': '750000', 'ran
k': 'Captain'}
{'name': 'Davy Jones', 'ship': 'The Flying Dutchman', 'bounty': '1000000', 'rank': '
k': 'Captain'}
{'name': 'Davy Jones', 'ship': 'The Flying Dutchman', 'bounty': '1000000', 'rank': '
{'name': 'Davy Jones', 'ship': 'The Flying Dutchman', 'bounty': '1000000', 'rank': '
Captain'}
{'name': 'Will Turner', 'ship': 'The Flying Dutchman', 'bounty': '300000', 'rank': '
{'name': 'Will Turner', 'ship': 'The Flying Dutchman', 'bounty': '300000', 'rank': '
First Mate'}
First Mate'}
{'name': 'Elizabeth Swann', 'ship': 'Interceptor', 'bounty': '200000', 'rank': 'Capt
ain'}
{'name': 'Joshamee Gibbs', 'ship': 'Black Pearl', 'bounty': '100000', 'rank': 'First Mate'}
{'name': 'Cutler Beckett', 'ship': 'HMS Endeavour', 'bounty': '0', 'rank': 'Commodore'}
```

CSV - Writing to a File

```python
import csv

with open('pirates.csv', mode='w') as file:
    writer = csv.writer(file, delimiter=',')

    writer.writerow(['Blackbeard', 'Queen Anne\'s Revenge', 1200000])
    writer.writerow(['Angelica Teach', 'Queen Anne\'s Revenge', 950000])
    writer.writerow(['James Norrington', 'Dauntless', 300000])
    writer.writerow(['Bartholomew Roberts', 'Royal Fortune', 850000])
    writer.writerow(['Calico Jack Rackham', 'Ranger', 400000])
```

Writing from a Dictionary to CSV

```
import csv

with open('pirates.csv', mode='w') as file:
    fieldnames = ['name', 'ship', 'bounty']  # Define column names
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()  # Write column names as the first row

    # Adding pirates to the CSV using dictionaries
    writer.writerow({'name': 'Blackbeard', 'ship': "Queen Anne's Revenge", 'bounty': 1
    writer.writerow({'name': 'Angelica Teach', 'ship': "Queen Anne's Revenge", 'bounty
    writer.writerow({'name': 'James Norrington', 'ship': 'Dauntless', 'bounty': 300000
    writer.writerow({'name': 'Bartholomew Roberts', 'ship': 'Royal Fortune', 'bounty':
    writer.writerow({'name': 'Calico Jack Rackham', 'ship': 'Ranger', 'bounty': 400000
```

Emoji Check

Do you feel you understand CSV encoding format and how to utilise it with Python? Say so if not!

1. 😢 Haven't a clue, please help!
2. 🙁 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 🙂 Yes, with team collaboration could try it
5. 😀 Yes, enough to start working on it collaboratively

Quiz

When data is encoded in CSV format, what do we call the character (such as a comma or tab), which is used to separate different fields within a record?

1. `separator`
2. `limiter`
3. `fielder`
4. `delimiter`

Quiz

You want to import the contents of a CSV file, and view the imported contents in dictionary format. Which of the following lines is likely to appear in your code?

1. `reader = csv.reader(file, delimiter=',')`
2. `csv_file = csv.DictReader(file)`
3. `writer = csv.writer(file, delimiter=',')`
4. `writer = csv.DictWriter(file, fieldnames=fieldnames)`

Exercise Time!

JSON - JavaScript Object Notation

- JavaScript Object Notation
- A file format that uses human-readable text to store and transmit data objects
- Also maps to multidimensional data
- Can hold any combination of objects using `{}` and lists using `[]`

JSON Data Representation

Name	Ship	Bounty
Blackbeard	Queen Anne's Revenge	1200000
Angelica Teach	Queen Anne's Revenge	950000
James Norrington	Dauntless	300000
Bartholomew Roberts	Royal Fortune	850000
Calico Jack Rackham	Ranger	400000

```
[
    {
        "name": "Blackbeard",
        "ship": "Queen Anne's Revenge",
        "bounty": 1200000
    },
    {
        "name": "Angelica Teach",
        "ship": "Queen Anne's Revenge",
        "bounty": 950000
    },
    {
        "name": "James Norrington",
        "ship": "Dauntless",
        "bounty": 300000
    },
    {
        "name": "Bartholomew Roberts",
        "ship": "Royal Fortune",
        "bounty": 850000
    },
    {
        "name": "Calico Jack Rackham",
        "ship": "Ranger",
        "bounty": 400000
    }
]
```

Before We Talk About JSON Building Blocks... Think About How Would You Describe Jack Sparrow?

Some Inspiration…

The Building Blocks of JSON

How would we describe Jack Sparrow?

Name: Jack Sparrow
Role: Pirate Captain
Ship: Black Pearl
Favorite Drink: Rum
Bounty: 1000000

The Building Blocks of JSON - Objects

Key **Value**

```
{
    "name": "Jack Sparrow",
    "role": "Pirate Captain",
    "ship": "Black Pearl",
    "favorite_drink": "rum",
    "bounty": 1000000
}
```

Adding More Details About Jack Sparrow

- In JSON, **arrays** are used to store multiple values in an ordered list.

```
{
    "name": "Jack Sparrow",
    "role": "Pirate Captain",
    "bounty": 1000000,
    "favorite_drink": "rum",
    "ships": ["Black Pearl", "Interceptor", "Queen Anne's Revenge"]
}
```

What About the Crew?

```
{
    "name": "Jack Sparrow",
    "role": "Pirate Captain",
    "bounty": 1000000,
    "ships": ["Black Pearl", "Interceptor", "Queen Anne's Revenge"],
    "crew": [
        {
            "name": "Gibbs",
            "role": "First Mate"
        },
        {
            "name": "Anamaria",
            "role": "Navigator"
        },
        {
            "name": "Elizabeth Swann",
            "role": "Pirate"
        }
    ]
}
```

JSON Example - Code

```python
# Reading the JSON data back into Python
with open('jack_sparrow.json', 'r') as json_file:
    jack_data = json.load(json_file)

# Print the data to verify
print("Name:", jack_data["name"])
print("Ship:", jack_data["ship"])
print("Allies:", [ally["name"] for ally in jack_data["allies"]])
```

```python
import json

# Our JSON object representing Jack Sparrow
jack_sparrow = {
    "name": "Jack Sparrow",
    "role": "Pirate Captain",
    "ship": "Black Pearl",
    "bounty": 5000000,
    "traits": ["Cunning", "Resourceful", "Unpredictable"],
    "allies": [
        {"name": "Will Turner", "role": "Blacksmith turned Pirate"},
        {"name": "Elizabeth Swann", "role": "Pirate Lord of the South China Sea"}
    ]
}

# Writing the JSON data to a file
with open('jack_sparrow.json', 'w') as json_file:
    json.dump(jack_sparrow, json_file) #mention indent=4 here after running

print("Jack Sparrow's data has been saved to jack_sparrow.json")
```

Python objects and their equivalent conversion to JSON

Python	JSON Equivalent
dict	object
str	string
list, tuple	array
int, float	number
True	true
False	false
None	null

JSON Advantages

- Human-readable
- Lightweight
- Language independence
- Nesting

JSON Disadvantages

- Lack of comments
- Only supports basic data types
- Less efficient for large datasets

Emoji Check

Do you feel you understand JSON encoding format and how to utilise it with Python? Say so if not!

1. 😢 Haven't a clue, please help!
2. 🙁 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 🙂 Yes, with team collaboration could try it
5. 😀 Yes, enough to start working on it collaboratively

Exercise Time!

YAML

- Stands for "YAML Ain't Markup Language"
- A human-readable data serialization format
- Uses indentation instead of brackets or commas
- Often used for configuration files, data exchange, and automation

YAML Features

YAML is structured using indentation rather than brackets and commas.

- Uses indentation instead of brackets {} and [ ]
- Lists are created using dashes (-)
- No need for quotation marks unless necessary
- Easy to read and write!

```yaml
pirates:
  - name: Jack Sparrow
    ship: Black Pearl
    bounty: 1000000
  - name: Blackbeard
    ship: Queen Anne's Revenge
    bounty: 1200000
  - name: Davy Jones
    ship: Flying Dutchman
    bounty: 1500000
```

JSON vs YAML

JSON

```
{
"employees":
    [
      {
       "id": "40159",
       "name": "John",
       "technology": "Cloud computing",
       "title": "Engineer",
       "team": "Development"
      }
    [
}
```

YAML

```
employees:
    - id: '40159'
      name: John
      technology: Cloud computing
      title: Engineer
      team: Development
```

Thank you! :)